

TP1 Microprocesseur START-UP

26/01/2026



Loic Weber

Mellyna Derridj

Questions de préparations :

1. Au reset le microcontrôleur est à l'adresse RESET_Vector, et la première instruction exécutée après le reset se trouve à l'adresse RESET_Vector+4.

Pour l'obtenir dans STM32, il faut:

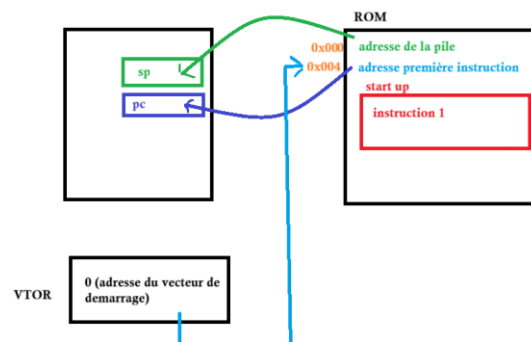
- Lancer le mode debugueur
- ouvrir la vue "SFR"
- Aller dans le registre VTOR (on y trouvera l'adresse de la table des vecteurs)
- aller à l'adresse VTOR + 4 qui est donc l'adresse recherchée
- Puis à partir de l'adresse on peut retrouver la première instruction en allant dans Memory view.

7. Start up, where does it start ?

On considère le code suivant :

```
The code is:
.global Reset_Handler
.text
Reset_Handler:
// 1:initializations to clear the stack
mov R6,#0
ldr R7,=0x20017FF0
str R6,[R7]
ADD R7,#4
str R6,[R7]
ADD R7,#4
str R6,[R7]
ADD R7,#4
str R6,[R7]
// 2:initialization of the stack pointer
ldr SP,=0x20018000 //(just after the RAM)
// 3:other initialization
ldr R0,=0x20000000 // at the beginning of the RAM
MOV R1,#1
STR R1,[R0]
AndAgain:
B AndAgain
```

Intérêt du code : la méthode reset handler est déjà définie par STM32 mais de façon “weak” on peut donc la redéfinir comme dans ce code.



Gestion de l'espace mémoire (simplifié) après un Reset :

VTOR est le vecteur de démarrage, son adresse est celle du registre des vecteurs. Il contient l'adresse du haut de la pile (en l'occurrence 0). Il pointe donc vers l'adresse de la pile (contenue sur 4 octets soit 32bits). C'est donc en 0x04 que se trouve l'adresse de la première instruction.

0x0 <Hex>	0x0: 0x0 <Traditional>	New Renderings...
0x00000000	20018000 080001C8 0800031D 08000325 0800032D 08000335 0800033D 00000000 00000000 00000000	... E.....%.....5...=.....
0x00000028	00000000 08000345 08000353 00000000 08000361 0800036F 0800037B 0800037B 0800037B 0800037B	E...S.....a...o...{...{...{...{...
0x00000050	0800037B 0800037B 0800037B 0800037B 0800037B 0800037B 0800037B 0800037B 0800037B 0800037B	{...{...{...{...{...{...{...{...{...{...
0x00000078	0800037B 0800037B 0800037B 0800037B 0800037B 0800037B 0800037B 0800037B 0800037B 0800037B	{...{...{...{...{...{...{...{...{...{...
0x000000A0	0800037B 0800037B 0800037B 0800037B 0800037B 0800037B 0800037B 0800037B 0800037B 0800037B	{...{...{...{...{...{...{...{...{...{...
0x000000C8	0800037B 0800037B 0800037B 0800037B 0800037B 0800037B 0800037B 0800037B 0800037B 0800037B	{...{...{...{...{...{...{...{...{...{...
0x000000F0	0800037B 0800037B 0800037B 0800037B 0800037B 0800037B 0800037B 0800037B 0800037B 0800037B	{...{...{...{...{...{...{...{...{...{...
0x00000118	0800037B 0800037B 0800037B 0800037B 0800037B 0800037B 0800037B 0800037B 0800037B 0800037B	{...{...{...{...{...{...{...{...{...{...
0x00000140	0800037B 0800037B 0800037B 0800037B 0800037B 0800037B 0800037B 0800037B 0800037B 0800037B	{...{...{...{...{...{...{...{...{...{...

Affichage du contenu de la mémoire (format TRADITIONAL i.e Big Endian)

On lit alors l'adresse de la première instruction à exécuter : 080001C8.

Dans l'affichage desassembled on constate la même chose.

Variables		Breakpoints	Expressions	Disassembly	Registers	Live Expressions	SFRs
080001C8							
08000190:	ldr	r3, [pc, #16]	@ (0x80001a4 <__do_global_dtors_aux+28>)				
08000192:	cbz	r3, 0x800019a	<__do_global_dtors_aux+18>				
08000194:	ldr	r0, [pc, #16]	@ (0x80001a8 <__do_global_dtors_aux+32>)				
08000196:	nop.w						
0800019a:	movs	r3, #1					
0800019c:	strb	r3, [r4, #0]					
0800019e:	pop	{r4, pc}					
080001a0:	movs	r4, r0					
080001a2:	movs	r0, #0					
080001a4:	movs	r0, r0					
080001a6:	movs	r0, r0					
080001a8:	lsls	r4, r4, #14					
080001aa:	lsrs	r0, r0, #32					
frame_dummy:							
080001ac:	push	{r3, lr}					
080001ae:	ldr	r3, [pc, #12]	@ (0x80001bc <frame_dummy+16>)				
080001b0:	cbz	r3, 0x80001ba	<frame_dummy+14>				
080001b2:	ldr	r1, [pc, #12]	@ (0x80001c0 <frame_dummy+20>)				
080001b4:	ldr	r0, [pc, #12]	@ (0x80001c4 <frame_dummy+24>)				
080001b6:	nop.w						
080001ba:	pop	{r3, pc}					
080001bc:	movs	r0, r0					
080001be:	movs	r0, r0					
080001c0:	movs	r0, r1					
080001c2:	movs	r0, #0					
080001c4:	lsls	r4, r4, #14					
080001c6:	lsrs	r0, r0, #32					
Reset_Handler:							
080001c8:	mov.w	r6, #0					
080001cc:	ldr	r7, [pc, #324]	@ (0x8000314 <YOURFUNCT+4>)				
080001ce:	str	r6, [r7, #0]					
080001d0:	add.w	r7, r7, #4					
080001d4:	str	r6, [r7, #0]					
080001d6:	add.w	r7, r7, #4					
080001da:	str	r6, [r7, #0]					
080001dc:	add.w	r7, r7, #4					
080001e0:	str	r6, [r7, #0]					
27	ldr	SP, 0x20018000	//(just after the RAM)				
080001e2:	ldr.w	sp, [pc, #308]	@ 0x8000318 <YOURFUNCT+8>				
29	ldr	R0, 0x20000000	// at the beginning of the RAM				
080001e6:	mov.w	r0, #536870912	@ 0x20000000				
30	MOV	R1, #1					

Taille de la plus grande RAM (variables + stack) : **96 Ko** (Cf : TD)

8 Basic uses of the Stack

On considère le code suivant :

```
#####  
.global STACKA  
.text  
STACKA:  
    MOV R4,#234  
    MOV R5,#235  
    MOV R6,#236  
  
    PUSH {R4}  
    PUSH {R5}  
    PUSH {R6}  
  
    MOV R4,#0  
    MOV R5,#0  
    MOV R6,#0  
  
    POP {R4}  
    POP {R5}  
    POP {R6}  
  
END_STACKA: B END_STACKA
```

Evolution du stack A :

The screenshot displays a debugger interface with three main panels. The left panel shows assembly code with line numbers 17 to 57. The middle panel shows the state of general registers (r0-r15), the program counter (pc), and other registers (lr, xpsr, d0-d3). The right panel shows the memory browser view for the stack area, displaying hexadecimal and decimal values.

Assembly Code (Left Panel):

```
17  MOV R0,#0  
18  Ldr R7,=0x20017FF0  
19  str R6,[R7]  
20  ADD R7,#4  
21  str R5,[R7]  
22  ADD R7,#4  
23  str R4,[R7]  
24  ADD R7,#4  
25  str R6,[R7]  
26  // 2:initialisation of the stack pointer  
27  Ldr SP,=0x20018000 // (just after the RAM)  
28  // 3:other initialisation  
29  Ldr R8,=0x20000000 // at the beginning of the RAM  
30  MOV R11,#1  
31  STR R1,[R8]  
32  B STACKA  
33  AndAgain:  
34  B AndAgain  
35  //  
36  //  
37  //  
38  .global STACKA  
39  .text  
40  STACKA:  
41  MOV R4,#234  
42  MOV R5,#235  
43  MOV R6,#236  
44  
45  PUSH {R4}  
46  PUSH {R5}  
47  PUSH {R6}  
48  
49  MOV R4,#0  
50  MOV R5,#0  
51  MOV R6,#0  
52  
53  POP {R4}  
54  POP {R5}  
55  POP {R6}  
56  
57  END_STACKA: B END_STACKA
```

General Registers (Middle Panel):

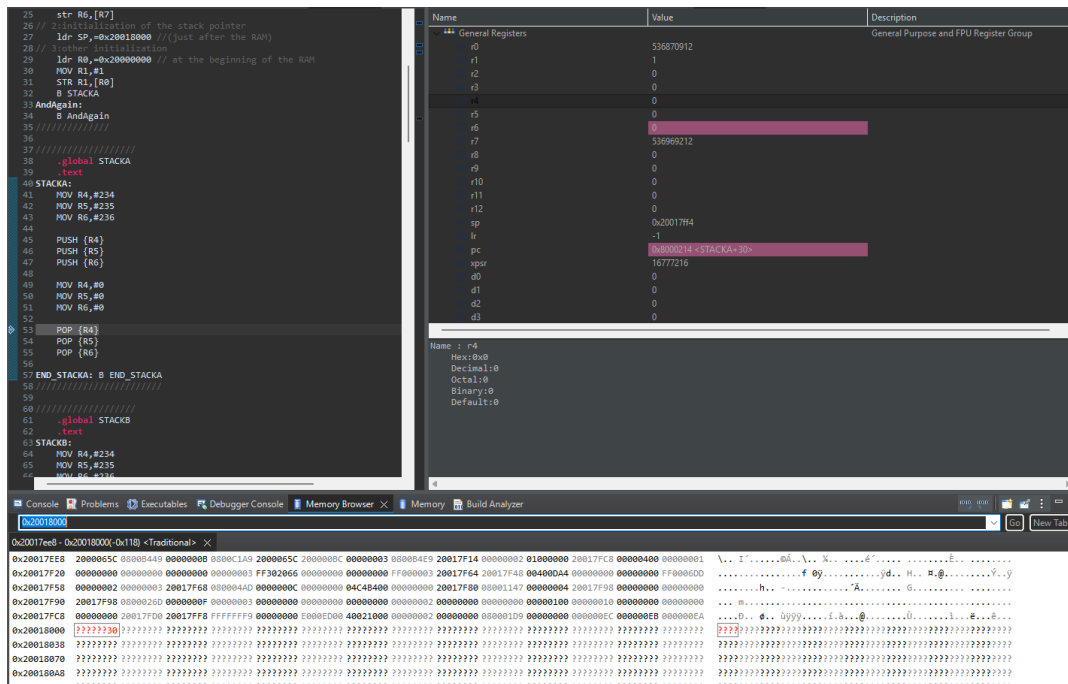
Name	Value	Description
General Registers		
r0	536870912	General Purpose and FPU Register Group
r1	1	
r2	0	
r3	0	
r4	234	
r5	235	
r6	236	
r7	536869212	
r8	0	
r9	0	
r10	0	
r11	0	
r12	0	
lr	0x20018000	
pc	-1	
xpsr	0x00000002 <STACKA+12>	
d0	16777216	
d1	0	
d2	0	
d3	0	

Memory Browser (Right Panel):

Name : sp
Hex:0x20018000
Decimal:536869216
Octal:04000300000
Binary:10000000000000110000000000000000
Default:0x20018000

The memory browser shows the stack area starting at 0x20018000. The stack is currently empty, with all memory locations containing 0x00000000.

Stack A avant le premier push



STACK A: après avoir clear le registre

Initial State before first PUSH	R4 R5 R6	0x 000000EA 0x 000000EB 0x 000000EC	SP	0x20018000	0x20017FF4> 0x00000000 0x20017FF8> 0x00000000 0x20017FFC> 0x00000000 0x20018000> 0x00000000
After PUSH {R4} PUSH {R5} and PUSH {R6}	R4 R5 R6	0x 000000EA 0x 000000EB 0x 000000EC	SP	0x20017FF4	0x20017FF4> 0x00000000 0x20017FF8> 0x00000000 0x20017FFC> 0x00000000 0x20018000> 0x00000000
After MOV R4, #0 MOV R5, #0 And MOV R6, #0	R4 R5 R6	0x 00000000 0x 00000000 0x 00000000	SP	0x20017FF4	0x20017FF4> 0x00000000 0x20017FF8> 0x00000000 0x20017FFC> 0x00000000 0x20018000> 0x00000000
After POP {R4} POP {R5} and POP {R6}	R4 R5 R6	0x 000000EA 0x 000000EB 0x 000000EC	SP	0x20018000	0x20017FF4> 0x00000000 0x20017FF8> 0x00000000 0x20017FFC> 0x00000000 0x20018000> 0x00000000

Evolution du STACK A

Le stack est descendant, le SP diminue lors d'un PUSH de 4 octets.

La pile A contient initialement les valeurs A B C (de haut en bas), à la fin elle contient C B A. L'ordre des éléments a été inversé donc cette méthode n'est pas la plus appropriée pour restaurer les registres.

On considère dorénavant le code suivant :

```
.global STACKB
.text

STACKB:
MOV R4,#234
MOV R5,#235
MOV R6,#236

PUSH {R4}
PUSH {R5}
PUSH {R6}

MOV R4,#0
MOV R5,#0
MOV R6,#0

POP {R6}
POP {R5}
POP {R4}

END_STACKB: B END_STACKB
```

Remarque : la seule différence avec le Stack A est l'ordre des POP.

The screenshot shows a debugger interface with two main panels. The left panel displays assembly code, and the right panel shows a register window.

Assembly Code (Left Panel):

```
51 MOV R6,#0
52
53 POP {R4}
54 POP {R5}
55 POP {R6}
56
57 END_STACKA: B END_STACKA
58
59
60
61 .global STACKB
62 .text
63 STACKB:
64 MOV R4,#234
65 MOV R5,#235
66 MOV R6,#236
67
68 PUSH {R4}
69 PUSH {R5}
70 PUSH {R6}
71
72 MOV R4,#0
73 MOV R5,#0
74 MOV R6,#0
75
76 POP {R6}
77 POP {R5}
78 POP {R4}
79
80 END_STACKB: B END_STACKB
81
82
83
84 .global Test_FUNC1
85 .text
86
87 Test_FUNC1:
88
89 MOV R4,#2 //first variable is 2
90 MOV R5,#6 //second variable is 6
91 BL FUNC1 //call of FUNC1
92
93
```

Register Window (Right Panel):

Name	Value	Description
General Registers		General Purpose and FPU Register Group
r0	536870912	
r1	1	
r2	0	
r3	0	
r4	0	
r5	0	
r6	0	
r7	536969212	
r8	0	
r9	0	
r10	0	
r11	0	
r12	0	
sp	0x20017ff4	
lr	-1	
pc	0x000023a <STACKB+30>	
xpsr	16777216	
d0	0	
d1	0	
d2	0	
d3	0	

Stack B : avant le premier push

```

52
53 POP {R4}
54 POP {R5}
55 POP {R6}
56
57 END_STACKA: B END_STACKA
58
59
60
61 .global STACKB
62 .text
63 STACKB:
64 MOV R4,#234
65 MOV R5,#235
66 MOV R6,#236
67
68 PUSH {R4}
69 PUSH {R5}
70 PUSH {R6}
71
72 MOV R4,#0
73 MOV R5,#0
74 MOV R6,#0
75
76 POP {R6}
77 POP {R5}
78 POP {R4}
79
80 END_STACKB: B END_STACKB
81
82
83
84
85 .global Test_FUNC1
86 .text
87 Test_FUNC1:
88
89 MOV R4,#2 //first variable is 2
90 MOV R5,#6 //second variable is 6
91 BL FUNC1 //call of FUNC1
92 ADD R4,R4,R5 //addition of the two variables
93 END Test_FUNC1: B END_Test_FUNC1

```

Name	Value	Description
General Registers		
r0	536870912	General Purpose and FPU Register Group
r1	1	
r2	0	
r3	0	
r4	234	
r5	235	
r6	236	
r7	536969212	
r8	0	
r9	0	
r10	0	
r11	0	
r12	0	
sp	0x20018000	
lr	-1	
pc	0x000023a <END_STACKB>	
xpsr	16777216	
d0	0	
d1	0	
d2	0	
d3	0	

Console Problems Executables Debugger Console Memory Browser Memory Build Analyzer

0x20018000

0x20017ee8 - 0x20018000 (0x118) <Traditional> X

0x20017EE8	2000065C 0800B449 00000000 0800C1A9 2000065C 200000BC 00000003 0800B4E9 20017F14 00000002 01000000 20017FC8 00000400 00000001	\\.. I'.....0A\\.. X..é'.....É.....
0x20017F20	00000000 00000000 00000000 00000003 FF302066 00000000 00000000 FF000003 20017F64 20017F48 004000A4 00000000 00000000 FF0000DD	f 0y.....yd.. H.. n.@.....ÿ..y
0x20017F58	00000002 00000003 20017F68 000004AD 0000000C 00000000 04C48400 00000000 20017F80 08001147 00000004 20017F98 00000000 00000000	h.'A..... G.....
0x20017F90	20017F98 0800026D 0000000F 00000003 00000000 00000000 00000000 00000002 00000000 00000000 00000100 00000010 00000000 00000000	... M.....
0x20017FC8	00000000 20017FD0 20017FF8 FFFFFFFF 00000000 E000ED00 40021000 00000002 00000000 080001D9 00000000 000000EC 000000EB 000000EA	...D.. 0. uygy....i.b...@.....û.....l...E...E...
0x20018000	???????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ???????	??
0x20018038	???????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ???????	??
0x20018070	???????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ???????	??
0x200180A8	???????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ???????	??
0x200180F0	???????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ???????	??

Stack B : après les 3 pop

```

51 MOV R6,#0
52
53 POP {R4}
54 POP {R5}
55 POP {R6}
56
57 END_STACKA: B END_STACKA
58
59
60
61 .global STACKB
62 .text
63 STACKB:
64 MOV R4,#234
65 MOV R5,#235
66 MOV R6,#236
67
68 PUSH {R4}
69 PUSH {R5}
70 PUSH {R6}
71
72 MOV R4,#0
73 MOV R5,#0
74 MOV R6,#0
75
76 POP {R6}
77 POP {R5}
78 POP {R4}
79
80 END_STACKB: B END_STACKB
81
82
83
84
85 .global Test_FUNC1
86 .text
87 Test_FUNC1:
88
89 MOV R4,#2 //first variable is 2
90 MOV R5,#6 //second variable is 6
91 BL FUNC1 //call of FUNC1
92 ADD R4,R4,R5 //addition of the two variables
93 END Test_FUNC1: B END_Test_FUNC1

```

Name	Value	Description
General Registers		
r0	536870912	General Purpose and FPU Register Group
r1	1	
r2	0	
r3	0	
r4	0	
r5	0	
r6	0	
r7	536969212	
r8	0	
r9	0	
r10	0	
r11	0	
r12	0	
sp	0x20017FF4	
lr	-1	
pc	0x000023a <STACKB+30>	
xpsr	16777216	
d0	0	
d1	0	
d2	0	
d3	0	

Console Problems Executables Debugger Console Memory Browser Memory Build Analyzer

0x20018000

0x20017ee8 - 0x20018000 (0x118) <Traditional> X

0x20017EE8	2000065C 0800B449 00000000 0800C1A9 2000065C 200000BC 00000003 0800B4E9 20017F14 00000002 01000000 20017FC8 00000400 00000001	\\.. I'.....0A\\.. X..é'.....É.....
0x20017F20	00000000 00000000 00000000 00000003 FF302066 00000000 00000000 FF000003 20017F64 20017F48 004000A4 00000000 00000000 FF0000DD	f 0y.....yd.. H.. n.@.....ÿ..y
0x20017F58	00000002 00000003 20017F68 000004AD 0000000C 00000000 04C48400 00000000 20017F80 08001147 00000004 20017F98 00000000 00000000	h.'A..... G.....
0x20017F90	20017F98 0800026D 0000000F 00000003 00000000 00000000 00000000 00000002 00000000 00000000 00000100 00000010 00000000 00000000	... M.....
0x20017FC8	00000000 20017FD0 20017FF8 FFFFFFFF 00000000 E000ED00 40021000 00000002 00000000 080001D9 00000000 000000EC 000000EB 000000EA	...D.. 0. uygy....i.b...@.....û.....l...E...E...
0x20018000	???????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ???????	??
0x20018038	???????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ???????	??
0x20018070	???????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ???????	??
0x200180A8	???????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ???????	??
0x200180F0	???????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ???????	??

Stack B après le clear du registre

Initial State before first PUSH	R4	0x 000000EA	SP	0x20018000	0x00000000
	R5	0x 000000EB			0x00000000
	R6	0x 000000EC			0x00000000
After PUSH {R4} PUSH {R5} and PUSH {R6}	R4	0x 0000EA	SP	0x20017FF4	0x 0 EC
	R5	0x 00 EB			0x 0 EB
	R6	0x 00 EC			0x 0 EA
After MOV R4, #0 MOV R5, #0 And MOV R6, #0	R4	0x 0	SP	0x20017FF4	0x 0 EC
	R5	0x 0			0x 0 EB
	R6	0x 0			0x 0 EA
After POP {R6} POP {R5} and POP {R4}	R4	0x 0 EA	SP	0x20018000	0x 0 EC
	R5	0x 0 EB			0x 0 EB
	R6	0x 0 EC			0x 0 EA

différence celles pop pas pareil

Evolution du stack B

La pile revient à l'état initial. Garantie donc la transparence d'une fonction car ne modifierait pas les valeurs. Si on ne prend pas le soin de sauvegarder les données des registres alors on perd ces données d'où l'intérêt de stack B.

On privilégiera la transparence de manière général.

9 Use of the Stack to insure transparency of registers R4 to R7

On considère le code suivant :

```
Test_FUNC1:
    MOV R4,#2 //first variable is 2
    MOV R5,#6 //second variable is 6
    BL FUNC1  //call of FUNC1 (should be transparent for variables)
    ADD R4,R4,R5 //add the variables (should be the same as before calling)
END_Test_FUNC1: B END_Test_FUNC1
```

```
FUNC1:
    MOV R4,#0 // That's it!
    MOV R5,#0 // Not very useful I must admit it!
    BX LR     //and no more (bye!)
```

You must modify your code, this way, to call the test (so at each time you will have to do the modification):

```
Reset_Handler:
    ldr SP,=0x20018000
    B Test_FUNC1
```

9.1

Lors de l'appel de fonction (BL FUNC1), le PC (program counter) contient l'adresse de cette instruction et LR (le link register également). Lors de l'exécution de la fonction FUNC1, une fois que le PC pointe vers l'adresse de l'instruction BX LR, LR prend comme valeur l'adresse de la dernière instruction de FUNC1.

L'adresse de retour est donc stockée dans le LR en l'occurrence : 0x8000253

Name	Value	Description
General Registers		
r0	0	General Purpose and FPU Register Group
r1	0	
r2	0	
r3	0	
r4	2	
r5	6	
r6	0	
r7	536969212	
r8	0	
r9	0	
r10	0	
r11	0	
r12	0	
sp	0x20018000	
lr	0x8000253 (Hex)	
pc	0x8000256 <FUNC1>	
xpsr	16777216	
d0	0	
d1	0	
d2	0	
d3	0	

Name	Value
lr	Hex:0x8000253 Decimal:134218323 Octal:01000001123 Binary:100000000000000000000001001010011 Default:134218323

FUNC1 n'est pas transparente car ne conserve pas les valeurs précédentes de R4 et

R5 (à savoir 2 et 6). Pour la rendre transparente il faudrait utiliser un programme de la forme de STACKB.

9.2

On considère le code suivant :

```
FUNCT2:
    PUSH {LR}
    MOV R4,#0
    MOV R5,#0
    POP {PC}
```

On revient à "ADD R4, R4, R5" exactement de la même manière qu'à la question précédente. LR a été stocké dans une pile et on finit avec l'adresse du bas de la pile dans LR (Rien ne change au niveau des valeurs de R4 et R5)
La fonction n'est toujours pas transparente.

9.3

On considère le code suivant :

```
FUNCT3:
    PUSH {R4,R5,LR}
    MOV R4,#0      // Stupid for a function
    MOV R5,#0      // I know!
    POP {R4,R5,PC} // BYE
```

Name	Value	Description
General Registers		
General Purpose and FPU Register Group		
r0	0	
r1	0	
r2	0	
r3	0	
r4	0	
r5	0	
r6	0	
r7	536969212	
r8	0	
r9	0	
r10	0	
r11	0	
r12	0	
sp	0x20017ff4	
lr	0x8000289 (Hex)	
pc	0x8000288 <FUNCT3+10>	
xpsr	16777216	
d0	0	
d1	0	
d2	0	
d3	0	

Avant le pop

Name	Value	Description
General Registers		
General Purpose and FPU Register Group		
r0	0	
r1	0	
r2	0	
r3	0	
r4	2	
r5	6	
r6	0	
r7	536969212	
r8	0	
r9	0	
r10	0	
r11	0	
r12	0	
sp	0x20018000	
lr	0x8000289 (Hex)	
pc	0x8000288 <Test_FUNCT3+12>	
xpsr	16777216	
d0	0	
d1	0	
d2	0	
d3	0	

Après le pop

On met les valeurs de R4, R5 et LR dans la pile et lors du dépileage, les valeurs de R4 et R5 pourront alors être restaurées.

La seule différence avec l'exemple 9.2 au niveau des valeurs est que R4 et R5 ne changent pas à l'issue de l'appel de fonction

Cette fonction est donc transparente, ce qui est un progrès au niveau de la sécurité du programme : les valeurs en mémoire d'une fonction transparente sont protégées.

10 Use of the Stack to insure nested calls

On considère le code suivant :

```
Test_FUNCT4_5:
    MOV R4,#2 //first variable is 2
    MOV R5,#6 //second variable is 6
    BL FUNCT4 //call of FUNCT4
    ADD R4,R4,R5 // addition of the two variables
END_Test_FUNCT4_5: B END_Test_FUNCT4_5

FUNCT4:
    PUSH {R4,R5,LR}
    MOV R4,#3
    MOV R5,#7
    BL FUNCT5

    ADD R4,R4,R5
    POP {R4,R5,PC}

FUNCT5:
    MOV R4,#0 //use of local variables
    MOV R5,#0 // why not!
    BX LR
```

10.1

"ADD R4,R4, R5" ne fonctionne toujours pas comme on le souhaiterait.

On pourrait régler ce problème en utilisant une pile dans R5 pour restaurer les valeurs à la sortie. (Pas sûr de ça à cause du fait qu'on en utilise déjà une, peut-être dépiler dans FUNCT5 ?)

Dans FUNCT4_5, ce sont les bonnes valeurs qui sont utilisées (grâce au pop qui

restaure les valeurs qu'on a conservé dans la pile).

10.2

BX LR ne peut pointer que sur une seule adresse, le premier pop (FUNCT7) fonctionne donc correctement puisque la dernière adresse enregistrée est la bonne, mais à la suite du deuxième (qui récupère cependant les bonnes valeurs), on ne retourne pas dans Test_FUNCT6_7, à la place on refait un tour dans FUNCT6 puisque l'adresse contenue dans BX LR n'est plus celle que l'on a récupéré dans Test_FUNCT6_7, dans lequel on ne peut donc plus retourner pour terminer la fonction.

11. A more evolved use of the Stack : FUNCTN

On considère le code suivant :

```
Test_FUNCTN:  
// R0 is the entering parameter of FUNCTN  
        MOV    R0,#5  
        BL     FUNCTN  
// after a call, R0 is expected to store the result  
// We copy it in R4:  
        MOV    R4,R0  
END_Test_FUNCTN: B END_Test_FUNCTN
```

The screenshot shows a debugger interface with two main panels. The left panel displays assembly code for a function named `Test_FUNCNT`. The right panel shows a register window with the following values:

Name	Value	Description
r0	5	General Purpose and FPU Register Group
r1	0	
r2	0	
r3	0	
r4	0	
r5	0	
r6	0	
r7	536969212	
r8	0	
r9	0	
r10	0	
r11	0	
r12	0	
sp	0x20017f80	
lr	0x80002f (Hex)	
pc	0x8000304 <FUNCTION+2>	
xpsr	16777216	
d0	0	
d1	0	
d2	0	
d3	0	

The assembly code in the left panel includes instructions like `PUSH {R4,R5}`, `MOV R4,R0`, `MOV R5,R0`, `POP {R4,R5}`, and `BX LR`. It also shows a `Test_FUNCNT` function definition and a `YOURFUNCTION` function definition.

Après le premier Push

The screenshot shows the same debugger interface as before, but with the assembly code and register window updated after the second `PUSH` instruction. The register window now shows:

Name	Value	Description
r0	4	General Purpose and FPU Register Group
r1	0	
r2	0	
r3	0	
r4	5	
r5	0	
r6	0	
r7	536969212	
r8	0	
r9	0	
r10	0	
r11	0	
r12	0	
sp	0x20017f80	
lr	0x8000313 (Hex)	
pc	0x8000304 <FUNCTION+2>	
xpsr	553648128	
d0	0	
d1	0	
d2	0	
d3	0	

The assembly code in the left panel is the same as before, but the `PUSH {R4,R5}` instruction is now highlighted, indicating it has been executed. The register window shows the updated values for `r0` (4) and `lr` (0x8000313).

Après le deuxième push

```

195 PUSH {R4,R5}
196 MOV R4,#0
197 MOV R5,#0
198 POP {R4,R5}
199 BX LR
200
201
202
203 .global Test_FUNCNT
204 .text
205 Test_FUNCNT:
206 // R0 is the entering parameter of FUNCNT
207 MOV R0,#5
208 BL FUNCNT
209 // after a call R0 is expected to store the result
210 // We copy it in R4:
211 MOV R4,R0
212 END_Test_FUNCNT: B END_Test_FUNCNT
213
214 .global FUNCNT
215 .type FUNCNT, %function
216 .text
217 FUNCNT: PUSH {R4,LR}
218 // R0 is the first entering parameter of FUNCNT
219 CNP R0,#1
220 BEQ RET_FUNCNT
221 MOV R4,R0
222 SUB R0,#1 // before a call R0 is the entering parameter
223 BL FUNCNT
224 MUL R0,R4 // after a call R0 is the return value
225 RET_FUNCNT:
226 POP {R4,PC}
227
228
229
230 .global YOURFUNCNT
231 .type YOURFUNCNT, %function
232 .text
233 YOURFUNCNT: PUSH {R4,LR}
234 // R0 is the first entering parameter of FUNCNT
235 // Insert your code here
236

```

Name	Value	Description
r0	3	General Purpose and FPU Register Group
r1	0	
r2	0	
r3	0	
r4	4	
r5	0	
r6	0	
r7	536969212	
r8	0	
r9	0	
r10	0	
r11	0	
r12	0	
sp	0x20017f68	
lr	0x8000313 (Hex)	
pc	0x8000304 <FUNCNT+2>	
xpsr	553648128	
d0	0	
d1	0	
d2	0	
d3	0	

Après le troisième push

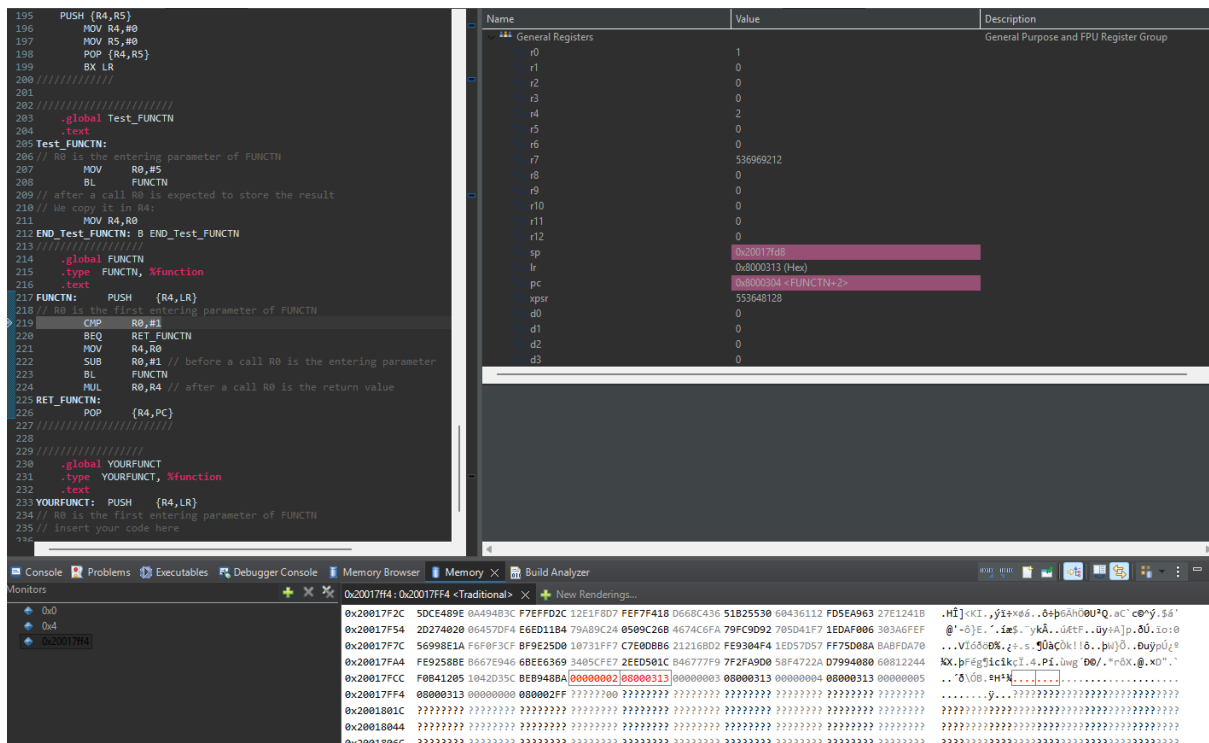
```

195 PUSH {R4,R5}
196 MOV R4,#0
197 MOV R5,#0
198 POP {R4,R5}
199 BX LR
200
201
202
203 .global Test_FUNCNT
204 .text
205 Test_FUNCNT:
206 // R0 is the entering parameter of FUNCNT
207 MOV R0,#5
208 BL FUNCNT
209 // after a call R0 is expected to store the result
210 // We copy it in R4:
211 MOV R4,R0
212 END_Test_FUNCNT: B END_Test_FUNCNT
213
214 .global FUNCNT
215 .type FUNCNT, %function
216 .text
217 FUNCNT: PUSH {R4,LR}
218 // R0 is the first entering parameter of FUNCNT
219 CNP R0,#1
220 BEQ RET_FUNCNT
221 MOV R4,R0
222 SUB R0,#1 // before a call R0 is the entering parameter
223 BL FUNCNT
224 MUL R0,R4 // after a call R0 is the return value
225 RET_FUNCNT:
226 POP {R4,PC}
227
228
229
230 .global YOURFUNCNT
231 .type YOURFUNCNT, %function
232 .text
233 YOURFUNCNT: PUSH {R4,LR}
234 // R0 is the first entering parameter of FUNCNT
235 // Insert your code here
236

```

Name	Value	Description
r0	2	General Purpose and FPU Register Group
r1	0	
r2	0	
r3	0	
r4	3	
r5	0	
r6	0	
r7	536969212	
r8	0	
r9	0	
r10	0	
r11	0	
r12	0	
sp	0x20017f68	
lr	0x8000313 (Hex)	
pc	0x8000304 <FUNCNT+2>	
xpsr	553648128	
d0	0	
d1	0	
d2	0	
d3	0	

Après le quatrième push



Après le cinquième push

(Requested work 14):FUNCTN est appelée 5 fois au total (5 passages dans la "boucle").

What is computation ?

A chaque passage dans la boucle, 2 octets de mémoires sont utilisés, donc pour R0=5, 10 octets sont utilisés. Pour R0=n, 2n octets seront donc utilisés.

Si la pile occupe 512 octets, on peut donc faire 256 passages dans la boucle sans dépasser l'espace mémoire et provoquer une erreur.

La plus grande valeur que j'ai pu mettre est 65538